

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY

HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY



Project Report

Embedded System

RFID ATTENDANCE SYSTEM

Instructor: BUI QUOC BAO

Semester: 252

Class: CC01

Group: 05

Group members: Huynh Tuan Anh – 2351001

Pham Duc Huy – 2351033

Ha Tuan Khoi – 2351045

Abstract

This report presents the design and implementation of an **RFID Attendance System** based on the STM32F411CEU6 microcontroller. The system is designed to identify users through RFID cards, display attendance information on an OLED screen, maintain accurate real-time timestamps using a DS3231 real-time clock module, and provide audio feedback through an I2S audio interface. The system also supports administrator-based card registration and deletion, where valid user identifiers are stored persistently in the internal Flash memory of the microcontroller.

The project integrates several hardware peripherals, including the MFRC522 RFID reader, SSD1306 OLED display, DS3231 RTC module, MAX98357A I2S audio amplifier, push button input, and optional MicroSD storage. The software is organized into a layered firmware architecture consisting of hardware abstraction, board support package drivers, application logic, and the main application layer. This structure improves modularity, maintainability, and reusability of the firmware.

The proposed system can perform RFID identification, user database lookup, timestamp acquisition, OLED display update, and voice feedback with low latency. The use of internal Flash enables persistent storage of registered cards, while RAM caching improves UID lookup speed. The system is compact, low-cost, and suitable for attendance monitoring in classrooms, laboratories, offices, and small access-control applications.

Keywords: RFID, attendance system, STM32F411CEU6, MFRC522, DS3231, OLED, I2S audio, embedded system.

Contents

Abstract i

1 Introduction 1

1.1 Project Background 1

1.2 Problem Statement 1

1.3 Objectives 2

1.4 Scope of the Report 2

1.5 Report Structure 2

2 Theoretical Background 3

2.1 Overview of an Embedded RFID Attendance System 3

2.2 RFID Technology 4

2.3 RFID UID and Authentication Concept 5

2.4 MFRC522 RFID Reader 5

2.5 STM32F411CEU6 Microcontroller 6

2.6 GPIO in Embedded Systems 7

2.7 SPI Communication 7

2.8 I2C Communication 8

2.9 OLED Display and SSD1306 Controller 9

2.10 Real-Time Clock and DS3231 Module 10

2.11 I2S Audio Interface 11

2.12 DMA for Audio Playback 11

2.13 WAV Audio Data in Embedded Systems 12

2.14 Internal Flash Memory 12

2.15 UID Database Design 13

2.16 RAM Cache for Fast Lookup 13

2.17	Flash Erase-Modify-Write Operation	14
2.18	MicroSD Storage	15
2.19	Button Debounce Theory	15
2.20	Embedded State Machine	16
2.21	Layered Firmware Architecture	17
2.22	Power Supply Theory	19
2.23	Security Considerations	19
2.24	Summary of Theoretical Background	20
3	System Overview and Design	21
3.1	General Description	21
3.2	System Block Diagram	21
3.3	Functional Requirements	22
3.4	Non-Functional Requirements	22
4	Hardware Design	24
4.1	Main Components	24
4.2	Power Supply Design	24
4.3	Schematic Overview	26
4.4	PCB Layout Overview	27
4.5	PCB Layout Overview	27
4.6	Pin Mapping	29
4.7	Hardware Design Assessment	29
5	Software Architecture	31
5.1	Layered Architecture	31
5.2	Layer 1: HAL, Third-Party Libraries, and Resources	32
5.3	Layer 2: BSP Driver Wrappers	32
5.4	Layer 3: Application Logic	32

5.5	Layer 4: Main Application	33
5.6	Folder Structure	33
6	Main Workflow and State Machine	35
6.1	Normal Attendance Mode	35
6.2	Administrator Mode	35
6.3	Admin Mode State Machine	35
6.4	Attendance Logic	37
7	Implementation Details	38
7.1	RFID Reading	38
7.2	UID Database in Flash	38
7.3	RAM Cache	38
7.4	Atomic Flash Update	39
7.5	OLED User Interface	39
7.6	RTC Timestamping	40
7.7	Voice Feedback	40
7.8	Button Debounce	41
8	Results and Evaluation	42
8.1	Implemented Features	42
8.2	Memory Usage	42
8.3	Timing Performance	43
8.4	Requirement Satisfaction	43
8.5	Overall Assessment	44
9	Discussion	45
9.1	Advantages	45
9.2	Limitations	45

9.3	Practical Applicability	46
9.4	Possible Improvements	46
10	Conclusion	47
	References	48
A	Appendix A: Suggested Figure Files	49
B	Appendix B: Main Pseudocode	49
C	Appendix C: Admin Mode Pseudocode	50

List of Figures

1	Overall block diagram of the RFID Attendance System	21
2	Power supply and current estimation diagram	25
3	System schematic	26
4	PCB layout after applying GND polygon pour on the top and bottom layers.	28
5	3D view of the completed RFID Attendance System PCB.	29
6	Four-layer firmware architecture of the RFID Attendance System	31
7	Administrator mode state machine	36

List of Tables

1	Comparison of firmware layers	18
2	Functional requirements	22
3	Main hardware components	24
4	Estimated current consumption	25
5	Pin mapping	29
6	Administrator mode state description	37
7	Implemented technical features	42
8	Memory usage	42
9	Timing performance	43
10	Requirement satisfaction	44
11	Suggested image file names	49

1. Introduction

1.1. Project Background

Attendance management is an important activity in schools, laboratories, offices, and many other organizations. Traditional attendance methods such as paper-based sign-in sheets or manual checking are simple but inefficient. They require human supervision, consume time, and may lead to mistakes such as missing records, duplicated entries, or inaccurate time information.

An embedded RFID attendance system provides a more reliable and automatic solution. Each user carries an RFID card containing a unique identifier. When the card is placed near the RFID reader, the system reads the card UID, checks whether the UID exists in the authorized database, records the attendance state, and provides feedback to the user. By combining RFID identification with real-time timestamping, visual display, and voice feedback, the system can reduce manual workload and improve the reliability of attendance records.

In this project, Group 5 developed an RFID Attendance System based on the STM32F411CEU6 Black Pill development board. The system uses an MFRC522 RFID reader for card detection, an SSD1306 OLED display for local user interface, a DS3231 real-time clock module for timekeeping, internal Flash memory for persistent UID database storage, and an I2S audio amplifier for voice feedback.

1.2. Problem Statement

The main problem addressed by this project is how to design a compact embedded system that can automatically identify users, manage attendance status, store authorized RFID card information, and provide clear feedback to the user.

The system must satisfy several practical requirements. First, the card identification process must be fast enough for real-time use. Second, the system must store registered users even after reset or power loss. Third, the user interface must be simple and understandable. Fourth, the firmware should be organized clearly so that each module has a specific responsibility and can be reused or modified independently. Finally, the system should support administrator operations such as adding or deleting cards without requiring the firmware to be reprogrammed.

1.3. Objectives

The main objectives of this report are:

- To present the hardware design of an RFID Attendance System based on the STM32F411CEU6 microcontroller.
- To explain the theoretical background of RFID, SPI, I2C, I2S, RTC, OLED display, Flash memory, and embedded state machines.
- To describe the firmware architecture using a four-layer structure.
- To analyze the administrator mode state machine for adding and deleting RFID cards.
- To evaluate the system in terms of functionality, memory usage, response time, modularity, and practical applicability.
- To discuss limitations and propose future improvements.

1.4. Scope of the Report

This report focuses on the design and analysis of the embedded hardware and firmware architecture. The main scope includes the STM32F411CEU6 microcontroller, RFID card reading, OLED display interface, RTC timestamp acquisition, internal Flash database storage, button-based administrator mode, and I2S voice feedback.

The report does not focus on large-scale server-based attendance management, wireless cloud synchronization, or mobile application development. These features are considered possible future extensions.

1.5. Report Structure

This report is organized as follows. Section 2 presents the theoretical background. Section 3 describes the overall system design. Section 4 explains the hardware design and peripheral connections. Section 5 presents the software architecture. Section 6 analyzes the main workflows and state machines. Section 7 discusses implementation details. Section 8 presents the results and system evaluation. Section 9 discusses limitations and future improvements. Section 10 concludes the report.

2. Theoretical Background

2.1. Overview of an Embedded RFID Attendance System

An RFID attendance system is an embedded system that uses radio-frequency identification to recognize users and record their attendance status. Instead of manually writing names or checking student IDs, each user is assigned an RFID card or tag. When the tag is placed near the reader, the reader obtains the tag identifier and sends it to the microcontroller. The microcontroller then checks whether the identifier belongs to a registered user. If the card is valid, the system records the attendance event and provides feedback through display, sound, or both.

A typical RFID attendance system contains four main parts:

- **Identification unit:** RFID card and RFID reader.
- **Processing unit:** microcontroller and firmware.
- **Storage unit:** internal Flash, external memory, or database.
- **User interface unit:** display, button, buzzer, or audio output.

In this project, the MFRC522 module is used as the RFID reader, STM32F411CEU6 is used as the main controller, internal Flash is used for storing the authorized UID database, and OLED plus voice audio are used as feedback interfaces. The DS3231 RTC module provides real-time information so that each attendance event can be associated with a meaningful timestamp.

The general operating concept can be summarized as follows:

1. A user scans an RFID card.
2. The RFID reader detects the card and reads its UID.
3. The UID is transferred to the STM32 microcontroller through SPI.
4. The firmware compares the UID with the registered UID database.
5. If the UID is valid, the system determines check-in or check-out status.
6. The RTC provides the current date and time.

7. The OLED shows the result and the audio module plays feedback.

This system is suitable for classroom or laboratory attendance because the interaction is simple, fast, and does not require direct physical contact between the card and the reader.

2.2. RFID Technology

Radio Frequency Identification, commonly known as RFID, is a wireless identification technology that uses electromagnetic fields to exchange data between an RFID reader and an RFID tag. An RFID system usually consists of three components: a reader, an antenna, and a tag. The reader generates an electromagnetic field through the antenna. When a passive RFID tag enters this field, the tag receives energy from the field and responds with stored data.

RFID tags can be classified into three main types:

- **Passive tags:** These tags do not contain an internal battery. They obtain power from the electromagnetic field generated by the reader. They are cheap, small, and commonly used in ID cards and attendance systems.
- **Active tags:** These tags contain a battery and can transmit signals over a longer range. They are more expensive and usually used in asset tracking or industrial systems.
- **Semi-passive tags:** These tags contain a battery for internal circuits but still rely on the reader signal for communication.

The RFID card used in this project is considered a passive contactless card. It is appropriate for an attendance system because the card does not require charging, the reading distance is short, and the cost is low.

RFID systems operate at different frequency bands. Common RFID frequency ranges include low frequency, high frequency, and ultra-high frequency. The MFRC522 module used in this project operates at 13.56 MHz, which belongs to the high-frequency RFID band. This frequency is widely used for contactless smart cards, student cards, access cards, and short-range identification systems.

2.3. RFID UID and Authentication Concept

The most basic information obtained from an RFID card is the UID, or Unique Identifier. A UID is a sequence of bytes used to identify a card. In a simple attendance system, the UID works similarly to a user ID. The firmware stores a list of authorized UIDs. When a card is scanned, the scanned UID is compared with this stored list.

The UID checking process can be represented as:

$$Result = \begin{cases} Authorized, & \text{if } UID_{scan} \in UID_{database} \\ Unauthorized, & \text{if } UID_{scan} \notin UID_{database} \end{cases} \quad (1)$$

This method is simple and fast. However, UID-based authentication has security limitations because some types of RFID cards may be cloned. Therefore, UID checking is suitable for basic attendance monitoring but may not be sufficient for high-security access control. For more secure systems, cryptographic card authentication, encrypted memory blocks, or server-side verification should be used.

In this project, UID-based identification is acceptable because the main goal is to design an educational embedded attendance system with local registration and attendance feedback.

2.4. MFRC522 RFID Reader

The MFRC522 is a highly integrated RFID reader/writer IC for contactless communication at 13.56 MHz. It supports ISO/IEC 14443A and is commonly used with MIFARE cards [2]. The module version of MFRC522 is widely used in embedded projects because it is low-cost, easy to interface, and supported by many software libraries.

The MFRC522 communicates with the microcontroller through a serial interface. In this project, SPI is selected. The typical MFRC522 SPI pins are:

- **SCK:** SPI clock from the microcontroller.
- **MOSI:** Data from microcontroller to MFRC522.
- **MISO:** Data from MFRC522 to microcontroller.
- **SDA/SS:** Slave select or chip select.

- **RST:** Reset pin.

The basic MFRC522 reading process includes:

1. Initialize the MFRC522 registers.
2. Turn on the antenna driver.
3. Check whether a new card is present.
4. Perform anti-collision to obtain the card UID.
5. Transfer the UID to the application layer.
6. Stop communication with the current card.

The anti-collision step is important because it allows the reader to correctly identify a card even when multiple cards are present in the field. In the context of this attendance project, users normally scan one card at a time, but the anti-collision function is still part of the standard RFID reading sequence.

2.5. STM32F411CEU6 Microcontroller

The STM32F411CEU6 is the main processing unit of the system. It belongs to the STM32F4 series and is based on the ARM Cortex-M4 core. According to the STM32F411xC/xE datasheet, this microcontroller supports up to 100 MHz operating frequency, up to 512 KB Flash memory, and 128 KB SRAM [1]. These specifications are sufficient for this project because the system needs to handle multiple peripherals, store small data, and provide responsive user interaction.

The STM32F411CEU6 performs the following tasks in the RFID Attendance System:

- Initializes and controls all peripherals.
- Communicates with the MFRC522 RFID reader through SPI.
- Communicates with the OLED display and DS3231 RTC through I2C.
- Sends digital audio data to the MAX98357A amplifier through I2S.
- Reads button input using GPIO.

- Stores and updates the UID database in internal Flash memory.
- Executes the attendance and administrator state machines.

The reason for selecting STM32F411CEU6 is that it offers a good balance between performance, memory capacity, peripheral availability, and cost. It also has enough communication interfaces for this project, including SPI, I2C, I2S, GPIO, DMA, and internal Flash programming capability.

2.6. GPIO in Embedded Systems

General Purpose Input/Output, or GPIO, pins are basic digital pins that can be configured as input or output. In this project, GPIO pins are used for chip select signals, reset signals, and button input.

For example, the MFRC522 module uses GPIO pins for chip select and reset. The button uses a GPIO input pin with pull-up configuration. When the button is not pressed, the input remains at logic HIGH. When the button is pressed, it is connected to ground and becomes logic LOW. The firmware detects this transition and interprets it as a button event.

GPIO is simple, but it is important in embedded system design because many control signals are not handled automatically by communication peripherals. Correct GPIO configuration ensures stable and predictable hardware behavior.

2.7. SPI Communication

Serial Peripheral Interface, or SPI, is a synchronous serial communication protocol used for high-speed communication between a master and one or more slave devices. In this project, the STM32 acts as the SPI master, while the MFRC522 and optional MicroSD card act as SPI slave devices.

A standard SPI bus uses four signals:

- **SCK:** Serial clock generated by the master.
- **MOSI:** Master Out Slave In, used to transmit data from master to slave.
- **MISO:** Master In Slave Out, used to transmit data from slave to master.
- **CS/SS:** Chip select or slave select, used to choose the active slave device.

SPI is full-duplex, meaning data can be transmitted and received at the same time. Every clock pulse shifts one bit out and one bit in. This makes SPI faster than many simple serial protocols.

In this RFID Attendance System, SPI is suitable for MFRC522 communication because:

- RFID UID reading requires quick data transfer.
- The wiring is simple and stable for short distances.
- SPI drivers are directly supported by STM32 HAL.
- Multiple SPI devices can be used if each device has its own chip select line.

The optional MicroSD card also uses SPI. Since both MFRC522 and MicroSD may use SPI-type communication, proper chip select management is necessary. Only one SPI slave should be active at a time. When one device is selected, the chip select pins of other devices must remain inactive.

2.8. I2C Communication

Inter-Integrated Circuit, or I2C, is a two-wire synchronous serial communication protocol. It uses only two signal lines:

- **SDA:** Serial data line.
- **SCL:** Serial clock line.

Unlike SPI, I2C supports multiple devices on the same bus using unique device addresses. This is useful when the system needs to communicate with several low-speed peripherals but has limited microcontroller pins.

In this project, the OLED display and DS3231 RTC share the same I2C bus. The microcontroller sends commands or reads data by addressing the correct device. Pull-up resistors are normally required on SDA and SCL lines because I2C uses open-drain or open-collector signaling.

The advantages of I2C in this project are:

- It reduces the number of required pins.
- It allows multiple devices on the same bus.
- It is suitable for low-to-medium speed peripherals.
- It is easy to organize in firmware using device addresses.

However, I2C is slower than SPI and is more sensitive to wiring length and pull-up resistor values. Therefore, it is best used for short-distance communication on the same PCB or within a small embedded module.

2.9. OLED Display and SSD1306 Controller

The OLED display in this project is a small 128x64 pixel display controlled by an SSD1306 controller. OLED displays are suitable for embedded systems because they are compact, have high contrast, and do not require a backlight.

The SSD1306 controller contains internal display RAM. The microcontroller does not directly control each pixel through separate wires. Instead, it sends commands and display data through I2C. The OLED library then updates the internal display buffer and transfers it to the SSD1306.

The OLED display is used to show important information such as:

- System ready message.
- RFID card detected message.
- Check-in or check-out status.
- Unknown card warning.
- Administrator mode status.
- Add-card or delete-card instructions.
- Error or timeout messages.

In an attendance system, the display plays an important role because it gives immediate visual feedback to the user. Without a display, the user may not know whether the card was accepted, rejected, added, or deleted.

2.10. Real-Time Clock and DS3231 Module

A real-time clock, or RTC, is a hardware module that keeps track of time even when the main microcontroller is reset or powered off. In an attendance system, accurate time is important because each attendance record should include a timestamp.

The DS3231 is a high-accuracy RTC module with an integrated temperature-compensated crystal oscillator. It communicates with the microcontroller through I2C and can maintain time using a backup battery [3]. This means that the system does not lose the current time when the main power is disconnected.

The DS3231 can provide information such as:

- Seconds.
- Minutes.
- Hours.
- Day.
- Date.
- Month.
- Year.

In firmware, the STM32 reads the DS3231 registers through I2C. The returned time values are commonly stored in BCD format, so the firmware may need to convert BCD values into decimal values before displaying or logging them.

A typical timestamp format used in this project is:

$$YYYY - MM - DD \quad HH : MM : SS \quad (2)$$

Using an external RTC is more reliable than using only the internal MCU timer. The internal timer depends on the MCU running continuously, while the DS3231 can keep time independently.

2.11. I2S Audio Interface

Inter-IC Sound, or I2S, is a digital audio communication protocol used to transfer audio samples between digital audio devices. It is commonly used between a microcontroller and an audio codec, DAC, or digital amplifier.

A typical I2S interface includes:

- **BCLK:** Bit clock.
- **WS/LRCLK:** Word select or left-right clock.
- **DIN/DOUT:** Serial audio data.

In this project, the STM32 sends audio data to a MAX98357A digital amplifier. The amplifier receives digital I2S data and drives a speaker. The audio feedback can be used to play messages such as greeting or goodbye sounds.

The advantage of I2S audio feedback is that it improves user interaction. The user does not need to look carefully at the OLED display to know whether the attendance operation succeeded. The audio message gives immediate confirmation.

2.12. DMA for Audio Playback

Direct Memory Access, or DMA, is a hardware feature that allows data to be transferred between memory and peripherals without continuous CPU intervention. In audio playback, DMA is especially useful because audio samples must be transmitted at a stable rate.

Without DMA, the CPU would need to continuously send audio samples to the I2S peripheral. This may block other tasks and cause unstable playback. With DMA, the CPU only starts the transfer, and the DMA controller moves audio data automatically.

In this system, DMA helps the audio module operate more smoothly. While audio is being played, the CPU can still update the display, check button events, or prepare for the next RFID scan.

2.13. WAV Audio Data in Embedded Systems

Audio feedback in this project can be stored as C header files, such as `in_wav.h` and `out_wav.h`. These files contain audio sample arrays converted from WAV files. Instead of reading audio from a file system during runtime, the audio samples are compiled directly into program memory.

This approach has several advantages:

- It is simple to implement.
- It does not require external storage for audio files.
- Audio playback can start quickly.
- The audio data is always available after boot.

However, storing audio in Flash increases program memory usage. Therefore, audio duration and sample rate should be selected carefully. Short voice messages are suitable for embedded systems because they provide useful feedback without consuming too much memory.

2.14. Internal Flash Memory

Flash memory is non-volatile memory, meaning that stored data remains available after power loss. The STM32F411CEU6 includes internal Flash memory for program storage. A portion of this Flash can also be reserved for user data, such as the RFID UID database.

In this project, the UID database is stored in a reserved Flash sector. The database contains valid card UIDs and possibly additional information such as status flags or checksum values.

Flash memory is different from RAM in several important ways:

- Flash is non-volatile, while RAM is volatile.
- Flash write operations are slower than RAM writes.
- Flash usually must be erased by sector before being rewritten.
- Flash has limited erase/write endurance.

Because of these limitations, the system should not write to Flash too frequently. Normal attendance scanning should only read the UID database. Flash writing should occur mainly when adding or deleting cards in administrator mode.

2.15. UID Database Design

The UID database is the data structure used to store authorized cards. A simple UID database may contain a magic number, number of stored cards, UID records, and checksum.

A possible structure is:

```
1 typedef struct {  
2     uint32_t magic;  
3     uint32_t count;  
4     uid_record_t records[MAX_CARDS];  
5     uint32_t checksum;  
6 } flash_db_t;
```

The fields have the following meanings:

- **magic:** A fixed value used to check whether the database has been initialized.
- **count:** The number of valid UID records.
- **records:** The array containing registered UIDs.
- **checksum:** A value used to detect possible data corruption.

The UID lookup process should be simple and fast. When a card is scanned, the system compares the scanned UID with each UID in the RAM cache. If a matching UID is found, the user is considered valid.

2.16. RAM Cache for Fast Lookup

Although the UID database is stored in Flash, the system can copy the database into RAM during startup. This RAM copy is called a cache. The main reason for using a RAM cache is performance.

Flash reading is possible, but RAM access is faster and simpler for repeated searching. In this project, UID lookup using RAM cache takes less than 1 ms. This makes the attendance process feel immediate to the user.

The startup sequence can be described as follows:

1. Read the Flash database header.
2. Check the magic number and checksum.
3. If valid, copy UID records into RAM.
4. If invalid, initialize an empty database.
5. Use the RAM cache for normal UID lookup.

When an administrator adds or deletes a card, the RAM cache is updated and then written back to Flash.

2.17. Flash Erase-Modify-Write Operation

Flash memory cannot usually be modified like normal RAM. To change stored data, the firmware often needs to erase an entire sector and then write the new data. This is called an erase-modify-write operation.

For example, when adding a new UID:

1. Read the current database from Flash.
2. Add the new UID in RAM.
3. Erase the Flash sector.
4. Write the updated database to Flash.
5. Verify that the data was written correctly.

This operation takes longer than normal UID lookup. However, it only happens during administrator operations, so it does not affect normal attendance speed.

A more reliable design may use two Flash sectors. One sector stores the current valid database, and the other sector stores the new database during update. After writing is complete, a version flag or validity flag can be changed. This method reduces the risk of data loss if power fails during a Flash update.

2.18. MicroSD Storage

The system schematic includes an optional MicroSD socket. MicroSD cards are useful for storing larger amounts of data compared with internal Flash. In this project, the internal Flash is more suitable for storing the UID database because the database is small and must be quickly available after boot. The MicroSD card is more suitable for storing attendance logs.

A possible attendance log format is:

```
1 UID,Name,Date,Time,Status
2 A1B2C3D4,Student_01,2026-05-10,08:01:25,IN
3 A1B2C3D4,Student_01,2026-05-10,11:30:10,OUT
```

Using MicroSD for log storage has several advantages:

- The storage capacity is much larger than internal Flash.
- Data can be exported easily to a computer.
- CSV files can be opened using spreadsheet software.
- Long-term attendance history can be stored.

However, MicroSD handling is more complex than internal Flash. It may require a file system such as FATFS, and write operations may consume relatively high current. Therefore, MicroSD logging can be considered an extension feature rather than the core UID authentication mechanism.

2.19. Button Debounce Theory

Mechanical buttons do not produce a perfectly clean digital transition. When a button is pressed or released, the metal contacts may bounce for a short time. This causes the signal to rapidly switch between HIGH and LOW before becoming stable. This phenomenon is called button bouncing.

If the firmware reads the GPIO pin directly without debounce, one physical button press may be interpreted as multiple presses. In this project, that would be a serious problem because the button is used to change administrator mode.

A simple software debounce method is:

1. Read the button state.
2. Wait for a short debounce time, such as 20 ms.
3. Read the button state again.
4. Accept the change only if the state remains stable.

The button module should generate only one event for one valid button press. This is called edge-trigger detection. For example, a falling edge can be detected when the previous state is HIGH and the current stable state is LOW.

2.20. Embedded State Machine

A state machine is a design model in which the system can be in one of several defined states. The system changes from one state to another based on events or conditions. State machines are very useful in embedded systems because they make program behavior clear and predictable.

In this project, the administrator mode is designed as a state machine. Instead of writing many nested if-else statements, the firmware defines states such as:

- **NORMAL**
- **ADD_AUTH**
- **ADD_WAIT**
- **DELETE_AUTH**
- **DELETE_WAIT**

Each state has a clear purpose. For example, in **ADD_AUTH**, the system waits for the administrator card. In **ADD_WAIT**, the system waits for the card that should be added. Timeout conditions are also used to return the system to normal mode if no action is taken.

The advantages of using a state machine include:

- Easier debugging.
- Clearer program structure.

- Lower risk of unexpected behavior.
- Easier expansion in future versions.

2.21. Layered Firmware Architecture

In embedded system development, firmware architecture plays an important role in making the project clear, maintainable, and reusable. A small embedded program can be written entirely in `main.c`, but this approach becomes difficult to maintain when the system contains many peripherals and application functions. Therefore, a layered firmware architecture is used to separate low-level hardware operations from high-level application logic.

In this RFID Attendance System, the firmware is designed using a four-layer architecture. The highest layer contains the application orchestration code, while the lowest layer contains HAL drivers, third-party libraries, and hardware-related resources. The lower a layer is, the closer it is to the hardware. The higher a layer is, the closer it is to the application behavior.

The four layers used in this project are:

- **Layer 4 – Application Layer:** This is the highest layer of the firmware. It contains `main.c`, which is responsible for system initialization, main loop execution, and state-machine routing. This layer does not directly handle low-level hardware communication. Instead, it calls functions from the application logic layer.
- **Layer 3 – APP / Business Logic Layer:** This layer contains the main logic of the attendance system. It includes modules such as `attendance`, `student_db`, `ui_oled`, `admin_mode`, and `audio_player`. These modules implement the actual behavior of the system, such as processing a scanned card, checking whether a UID is valid, displaying messages, managing administrator mode, and controlling audio feedback.
- **Layer 2 – BSP / Board Support Package Layer:** This layer acts as a wrapper around low-level drivers. It provides project-specific APIs that are easier for the application layer to use. For example, instead of calling SPI functions directly in the attendance logic, the application can call a higher-level function such as `RFID_ReadUID(uid)`. This hides the hardware communication details from the business logic.

- **Layer 1 – HAL, Third-party Libraries, and Resources:** This is the lowest layer of the firmware. It contains STM32 HAL drivers, third-party libraries such as MFRC522 and SSD1306, and embedded resources such as `in_wav.h` and `out_wav.h`. This layer is closest to the hardware and provides basic functions for SPI, I2C, GPIO, I2S, DMA, and Flash access.

Table 1 compares the role of each layer in this project.

Table 1: Comparison of firmware layers

Layer	Role	Example in This Project
Layer 1: HAL	Direct hardware access and peripheral control	<code>HAL_SPI_Transmit()</code> , <code>HAL_I2C_Master_Receive()</code> , GPIO, DMA, I2S functions
Layer 2: BSP	Wrap HAL and third-party drivers into friendly APIs	<code>RFID_ReadUID(uid)</code> , <code>RTC_ReadTime()</code> , <code>FlashDB_Save()</code>
Layer 3: APP	Business logic independent of hardware details	<code>Attendance_ProcessCard(uid)</code> , <code>AdminMode_HandleCard(uid)</code> , <code>AudioPlayer_PlayIn()</code>
Layer 4: Application	System orchestration, initialization, and main loop control	<code>main.c</code> : initializes modules and routes the main state machine

This layered architecture brings several advantages. First, it improves readability because each module has a clear responsibility. Second, it improves maintainability because hardware-related changes can be handled in the BSP layer without modifying the attendance logic. Third, it improves reusability because low-level drivers can be reused in other STM32 projects. Finally, it reduces software coupling and prevents the firmware from becoming a single large and difficult-to-debug program.

Therefore, the four-layer firmware architecture is not only a folder organization method but also an important design principle of the RFID Attendance System.

2.22. Power Supply Theory

The system is powered from a 5 V USB adapter. Most modules in this project require 3.3 V, so a voltage regulator is used to convert 5 V to 3.3 V. The AMS1117-3.3 is a common linear regulator used in embedded projects.

For a linear regulator, the approximate power dissipation can be calculated as:

$$P_{loss} = (V_{in} - V_{out}) \times I_{load} \quad (3)$$

where V_{in} is the input voltage, V_{out} is the output voltage, and I_{load} is the load current.

For this project:

$$V_{in} = 5V, \quad V_{out} = 3.3V \quad (4)$$

Therefore, the voltage drop is:

$$V_{drop} = 5V - 3.3V = 1.7V \quad (5)$$

If the load current is approximately 265 mA, the regulator loss is:

$$P_{loss} = 1.7V \times 0.265A \approx 0.45W \quad (6)$$

This means the regulator may become warm during operation. Therefore, current consumption, heat dissipation, and stable decoupling capacitors should be considered in the hardware design.

2.23. Security Considerations

The basic version of this system uses UID comparison for user identification. This method is simple and suitable for an attendance prototype, but it is not a complete security solution. Some RFID cards can be copied, and UID-only checking does not verify whether the card is genuine.

To improve security, future designs may include:

- Admin card authentication.

- Password or keypad confirmation for administrator mode.
- Cryptographic RFID cards.
- Encrypted database storage.
- Server-side verification.
- Event logging to detect suspicious behavior.

In this project, administrator authentication is already included for card addition and deletion. This prevents ordinary users from modifying the UID database. However, stronger security methods would be required for high-security access control systems.

2.24. Summary of Theoretical Background

The RFID Attendance System combines several important embedded system concepts. RFID technology provides contactless identification. The STM32F411CEU6 microcontroller controls the system and communicates with multiple peripherals. SPI is used for RFID and optional MicroSD communication. I2C is used for OLED and RTC modules. I2S and DMA are used for efficient audio playback. Internal Flash provides persistent UID storage, while RAM caching provides fast lookup speed. The administrator state machine makes the system behavior organized and reliable.

These theoretical foundations support the design and implementation of a compact, modular, and practical attendance system.

3. System Overview and Design

3.1. General Description

The proposed system is an RFID Attendance System with visual and audio feedback. When an RFID card is scanned, the system reads the UID using the MFRC522 reader. The UID is then compared with the authorized UID list stored in Flash memory and cached in RAM. If the card is valid, the system determines whether the user is checking in or checking out. The system then reads the current time from the DS3231 RTC, updates the OLED display, and plays a corresponding voice message.

The system also supports an administrator mode. By pressing a physical button, the system can enter card adding or deleting mode. To prevent unauthorized modification of the UID database, the administrator must scan an admin card before adding or deleting another card.

3.2. System Block Diagram

Figure 1 shows the overall hardware block diagram of the system.

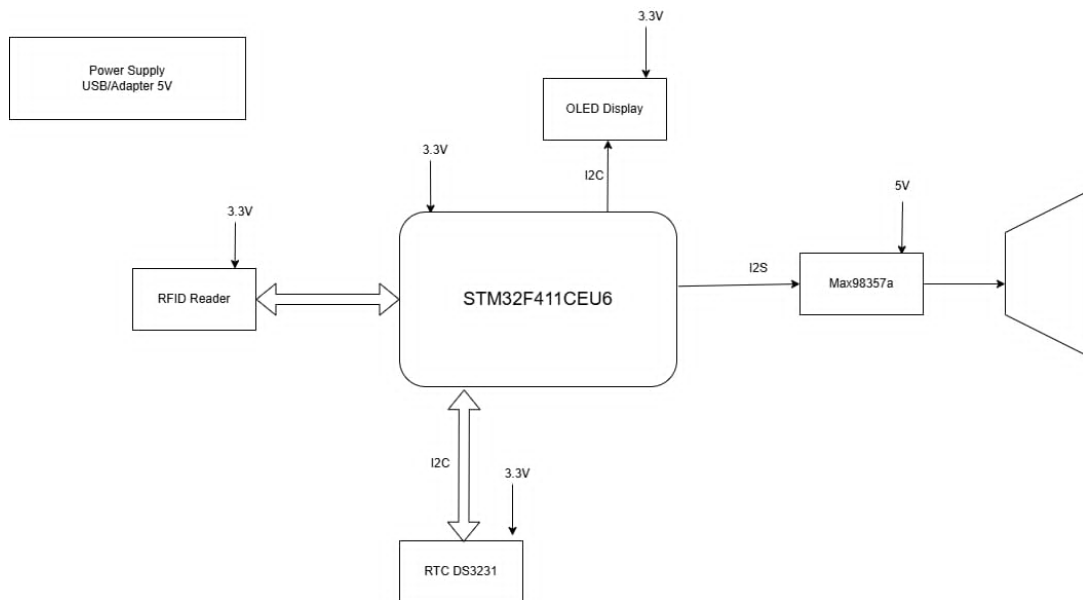


Figure 1: Overall block diagram of the RFID Attendance System

The STM32F411CEU6 is placed at the center of the system. The RFID reader communicates with the MCU through SPI. The OLED display and DS3231 RTC communicate

through I2C. The MicroSD socket is connected through SPI and can be used for extended attendance log storage. The I2S audio amplifier is connected to the MCU for voice playback.

3.3. Functional Requirements

The main functional requirements of the system are shown in Table 2.

Table 2: Functional requirements

No.	Requirement	Description
1	RFID card scanning	The system shall detect and read RFID card UID using the MFRC522 module.
2	User authentication	The system shall compare the scanned UID with the stored UID database.
3	Attendance logic	The system shall determine check-in or check-out status.
4	Timestamping	The system shall read real-time information from the DS3231 RTC.
5	Display feedback	The system shall display user and system status on the OLED screen.
6	Voice feedback	The system shall play corresponding audio feedback for successful attendance events.
7	Administrator mode	The system shall allow authorized adding and deleting of RFID cards.
8	Persistent storage	The system shall preserve the UID database after reset or power loss.
9	Optional log storage	The system may store attendance logs on a MicroSD card.

3.4. Non-Functional Requirements

Besides functional requirements, the system should also satisfy several non-functional requirements:

- **Low latency:** RFID reading and UID lookup should be fast enough for real-time user interaction.

- **Reliability:** The system should avoid corrupting the UID database during Flash update.
- **Modularity:** Each software module should have one clear responsibility.
- **Maintainability:** The firmware should be organized so that future modifications are easy.
- **Compactness:** The hardware should be simple and suitable for small embedded applications.
- **User friendliness:** The system should provide both display and audio feedback.

4. Hardware Design

4.1. Main Components

The main hardware components of the system are listed in Table 3.

Table 3: Main hardware components

Component	Interface	Function
STM32F411CEU6 Black Pill	Main MCU	Controls all peripherals and executes firmware logic.
MFRC522 RFID reader	SPI1	Reads RFID card UID.
SSD1306 OLED 128x64	I2C1	Displays status, UID result, and attendance information.
DS3231 RTC	I2C1	Provides accurate real-time date and time.
MAX98357A audio amplifier	I2S2 + DMA	Plays voice feedback.
Physical button	GPIO input	Switches between normal and administrator modes.
Internal Flash	MCU memory	Stores UID database persistently.
MicroSD socket	SPI3	Optional external storage for attendance logs.
AMS1117-3.3	Power regulator	Converts 5 V input to 3.3 V supply.

4.2. Power Supply Design

The system is powered by a 5 V USB adapter. Since most modules operate at 3.3 V, the system uses an AMS1117-3.3 low-dropout regulator to generate the 3.3 V rail. The 3.3 V rail supplies the STM32F411CEU6, MFRC522 RFID reader, OLED display, DS3231 RTC, and MicroSD socket.

Figure 2 shows the power distribution diagram.

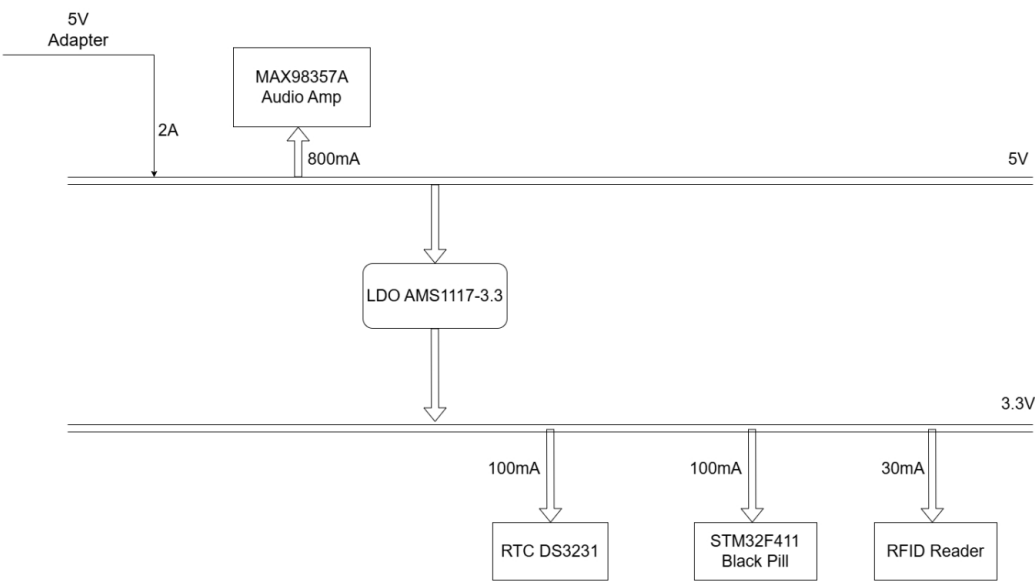


Figure 2: Power supply and current estimation diagram

The estimated current consumption is summarized in Table 4.

Table 4: Estimated current consumption

Component	Estimated Current	Note
DS3231 RTC	5 mA	Low-power timekeeping module.
OLED display	30 mA	Depends on brightness and displayed pixels.
MAX98357a	800 mA	Higher current during read-/write operation.
STM32F411 Black Pill	100 mA	Depends on clock and peripheral usage.
RFID reader	30 mA	Active during card scanning.
Total estimated current	965 mA	Excluding audio peak current.

The 5 V, 2 A adapter provides sufficient current margin for the system. However, the voltage regulator may dissipate heat because the voltage is dropped from 5 V to 3.3 V.

Therefore, current consumption and thermal behavior should be considered during long-term operation.

4.3. Schematic Overview

Figure 3 shows the complete schematic of the system.

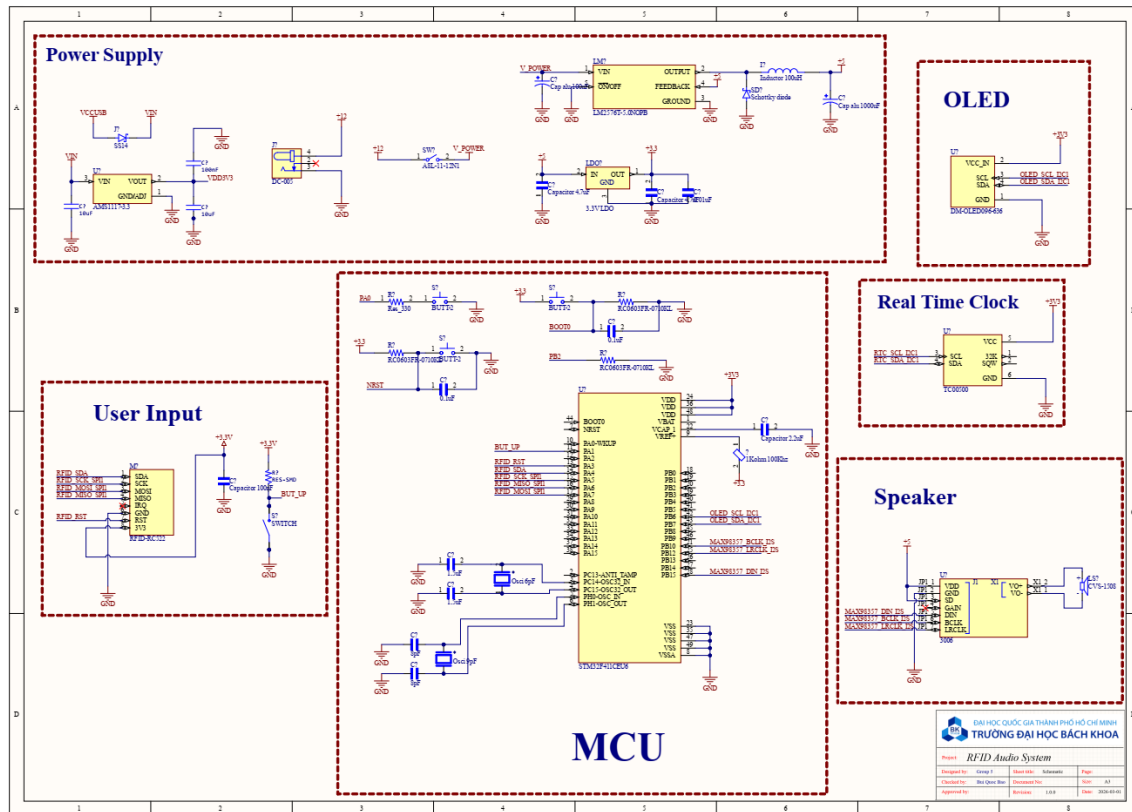


Figure 3: System schematic

The schematic is divided into several functional blocks:

- **Power Supply:** Converts USB 5 V to 3.3 V.
- **MCU:** Contains the STM32F411CEU6, crystal oscillator, reset circuit, boot configuration, and decoupling capacitors.
- **User Input:** Contains the RFID reader and button input.
- **Display:** Contains the OLED display connected through I2C.
- **Storage:** Contains the RTC module and MicroSD socket.

4.4. PCB Layout Overview

- **Power Supply:** Converts USB 5 V to 3.3 V.
- **MCU:** Contains the STM32F411CEU6, crystal oscillator, reset circuit, boot configuration, and decoupling capacitors.
- **User Input:** Contains the RFID reader and button input.
- **Display:** Contains the OLED display connected through I2C.
- **Storage:** Contains the RTC module and MicroSD socket.

4.5. PCB Layout Overview

The PCB layout was designed after completing the schematic verification and component footprint assignment. The main objective of the PCB layout is to transform the schematic circuit into a practical two-layer printed circuit board that can support stable power distribution, short signal paths, and organized placement of the main functional blocks.

The PCB was designed using a two-layer structure. The top layer is mainly used for component placement and signal routing, while the bottom layer is used for additional routing and ground copper pouring. Ground polygons were also used to improve the ground connection between modules and reduce the amount of manually routed ground traces. This approach helps simplify routing and provides a lower impedance return path for signals.

The STM32F411CEU6 microcontroller is placed near the center of the PCB because it is the main processing unit and connects to most peripherals. This placement reduces the average connection length between the MCU and external modules. The crystal oscillator and its load capacitors are placed close to the MCU oscillator pins to reduce noise sensitivity and improve clock stability. Decoupling capacitors are placed near the power pins of the MCU and other modules to stabilize the supply voltage.

The power supply block is placed near the input connector. The 5 V input, voltage regulator, diode, inductor, and filtering capacitors are arranged close to each other to keep the power path short. The 3.3 V and 5 V power traces are routed with wider tracks than normal signal traces because they carry current for multiple modules. Signal traces such as SPI, I2C, I2S, button lines, and control signals are routed with smaller widths suitable for digital communication.

The RFID module, OLED display, RTC module, audio amplifier, speaker connector, and user buttons are placed around the MCU according to their functional roles. The OLED and RTC share the I2C bus, so their routing is kept relatively direct. The MAX98357A audio amplifier and speaker are placed near each other to reduce the output trace length. The RFID module is placed separately from the audio and power sections to reduce possible interference.

The PCB layout uses vias to change between the top and bottom layers when signal crossing cannot be avoided. Ground connections are mainly completed through the ground polygon pour instead of manual point-to-point routing. After routing, Design Rule Check was performed to verify important electrical rules such as clearance, short circuit, track width, and unrouted nets. Minor silkscreen or mechanical clearance warnings may remain because the PCB is mainly designed for educational demonstration rather than immediate manufacturing.

Overall, the PCB layout successfully converts the system schematic into a compact physical board design. The layout keeps the main controller at the center, separates the power and peripheral blocks, uses wider traces for power distribution, and applies ground copper pour to improve signal return paths.

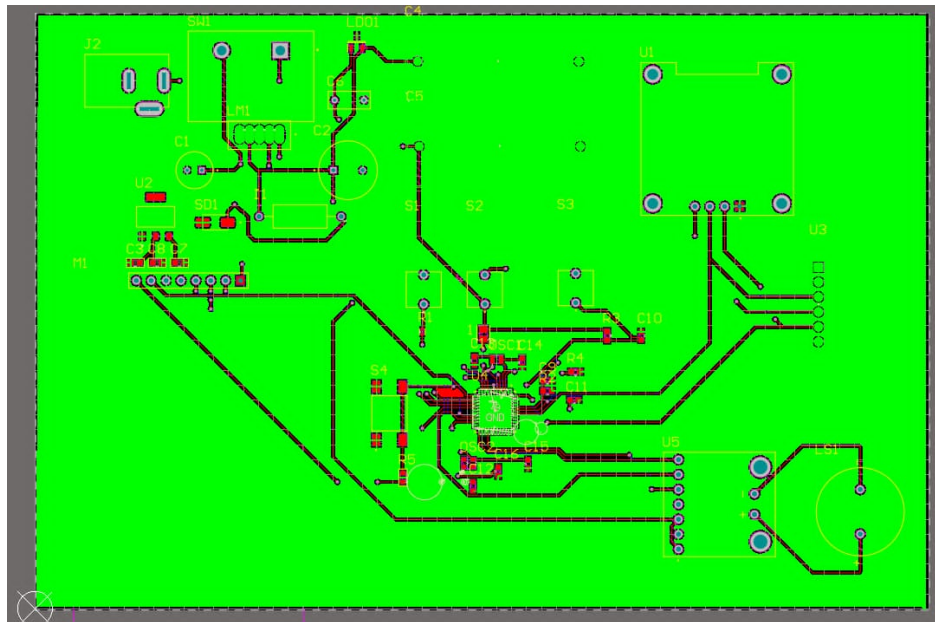


Figure 4: PCB layout after applying GND polygon pour on the top and bottom layers.

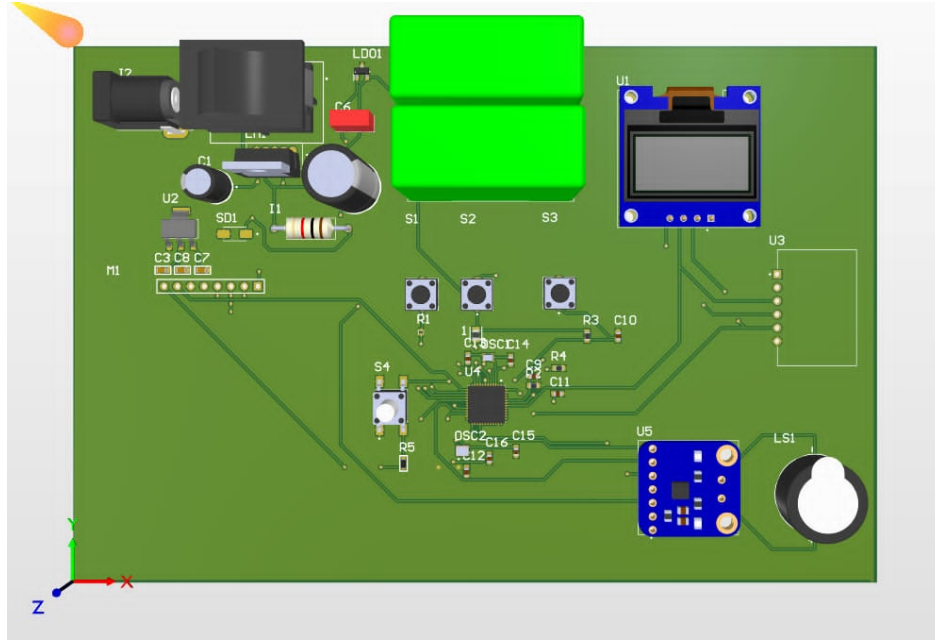


Figure 5: 3D view of the completed RFID Attendance System PCB.

4.6. Pin Mapping

The pin mapping of the system is shown in Table 5.

Table 5: Pin mapping

Function	Pin	Peripheral
RFID SCK / MISO / MOSI	PA5 / PA6 / PA7	SPI1
RFID RST / CS	PA3 / PA4	GPIO
OLED + DS3231 SCL / SDA	PB6 / PB7	I2C1
Button	PA1	GPIO input pull-up
Audio BCLK / WS / DIN	PB10 / PB12 / PB15	I2S2 + DMA
MicroSD SCK / MISO / MOSI / CS	SPI3 pins	SPI3

4.7. Hardware Design Assessment

The hardware design is suitable for a compact attendance system. Using I2C for both OLED and RTC saves GPIO pins. SPI is appropriate for the RFID reader because it provides fast communication and simple wiring. The optional MicroSD socket allows the system to be extended for long-term data logging.

The main limitation of the hardware is that multiple peripherals increase wiring complexity and require careful power supply design. The MicroSD module and audio amplifier can create current peaks, so decoupling capacitors and stable power regulation are important.

5. Software Architecture

5.1. Layered Architecture

The firmware is organized into four layers, as shown in Figure 6. This architecture separates low-level drivers from high-level application logic.

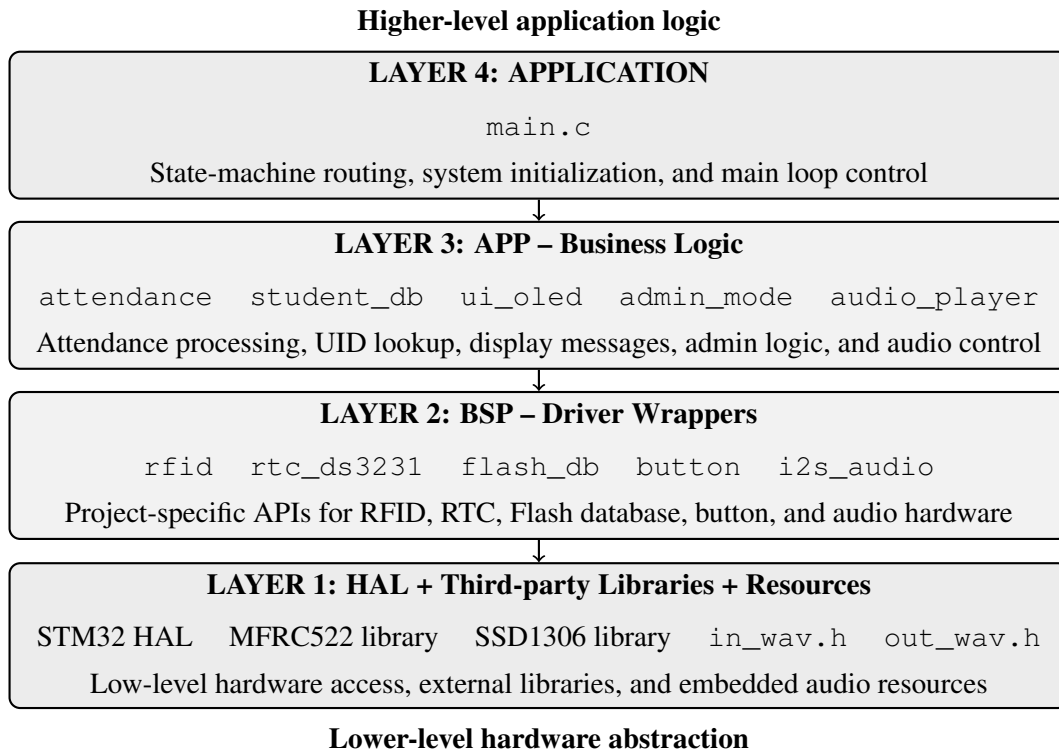


Figure 6: Four-layer firmware architecture of the RFID Attendance System

The four layers are:

- **Layer 1: HAL, third-party libraries, and resources**
- **Layer 2: BSP driver wrappers**
- **Layer 3: Application logic**
- **Layer 4: Main application and state machine**

This structure makes the firmware easier to maintain. If a hardware module is changed, only the corresponding BSP driver needs to be modified. The high-level application logic can remain mostly unchanged.

5.2. Layer 1: HAL, Third-Party Libraries, and Resources

Layer 1 contains the STM32 HAL library, third-party component libraries, and audio resources. The STM32 HAL provides low-level access to peripherals such as SPI, I2C, GPIO, DMA, and I2S. Third-party libraries include the MFRC522 driver and SSD1306 OLED driver. Audio resources such as `in_wav.h` and `out_wav.h` store the voice data used for feedback.

5.3. Layer 2: BSP Driver Wrappers

Layer 2 contains board support package modules. These modules wrap low-level HAL or third-party functions into simpler project-specific APIs.

The BSP modules include:

- `rfid`: reads UID from the MFRC522 module.
- `rtc_ds3231`: reads and writes time information from the DS3231 RTC.
- `flash_db`: manages the UID database in internal Flash memory.
- `button`: handles debounce and edge-trigger detection.
- `i2s_audio`: transmits audio data using I2S and DMA.

5.4. Layer 3: Application Logic

Layer 3 contains the business logic of the attendance system. This layer does not directly control hardware registers. Instead, it calls functions provided by the BSP layer.

The main modules in this layer are:

- `attendance`: determines check-in and check-out logic.
- `student_db`: performs UID lookup.
- `ui_oled`: prepares and displays messages on OLED.
- `admin_mode`: handles adding and deleting cards.
- `audio_player`: plays high-level voice feedback.

5.5. Layer 4: Main Application

Layer 4 contains the main program file. It initializes all hardware peripherals, loads the UID database from Flash into RAM, and repeatedly executes the main state machine.

The main loop performs the following tasks:

1. Check button event.
2. Update administrator mode state.
3. Poll RFID reader for card scan.
4. Process attendance or administrator operation.
5. Update OLED display.
6. Trigger audio feedback if needed.

5.6. Folder Structure

The project folder structure is shown below.

```
1 Core/
2 |-- Inc/
3 |   |-- BSP/
4 |   |   |-- rfid.h
5 |   |   |-- rtc_ds3231.h
6 |   |   |-- flash_db.h
7 |   |   |-- button.h
8 |   |   |-- i2s_audio.h
9 |   |
10 |   |-- APP/
11 |   |   |-- attendance.h
12 |   |   |-- student_db.h
13 |   |   |-- ui_oled.h
14 |   |   |-- admin_mode.h
15 |   |   |-- audio_player.h
16 |   |
17 |   |-- Resources/
18 |   |   |-- in_wav.h
```

```
19 | | |-- out_wav.h
20 | |
21 | |-- Third_Party/
22 | | |-- MFRC522.h
23 | | |-- ssd1306.h
24 | | |-- ssd1306_fonts.h
25 | | |-- ssd1306_conf.h
26 |
27 |-- Src/
28 | |-- BSP/
29 | |-- APP/
30 | |-- main.c
```

This organization clearly separates generated files, board-level drivers, application modules, third-party libraries, and audio resources.

6. Main Workflow and State Machine

6.1. Normal Attendance Mode

In normal mode, the system waits for an RFID card. When a card is detected, the system reads the UID and searches for it in the UID database.

The normal attendance workflow is:

1. The user scans an RFID card.
2. The MFRC522 reader reads the UID.
3. The UID is compared with the cached UID list in RAM.
4. If the UID is valid, the system reads the current time from the DS3231 RTC.
5. The attendance module determines whether the event is check-in or check-out.
6. The OLED display shows the result.
7. The audio module plays the corresponding voice feedback.

If the UID is not found in the database, the system displays an unauthorized card message and does not update the attendance status.

6.2. Administrator Mode

The administrator mode allows authorized modification of the UID database. This is necessary because new user cards may need to be added, and old cards may need to be removed.

To prevent unauthorized changes, the system requires an admin card authentication step. Only after the admin card is accepted can the system add or delete a target card.

6.3. Admin Mode State Machine

The administrator state machine is shown in Figure [7](#).

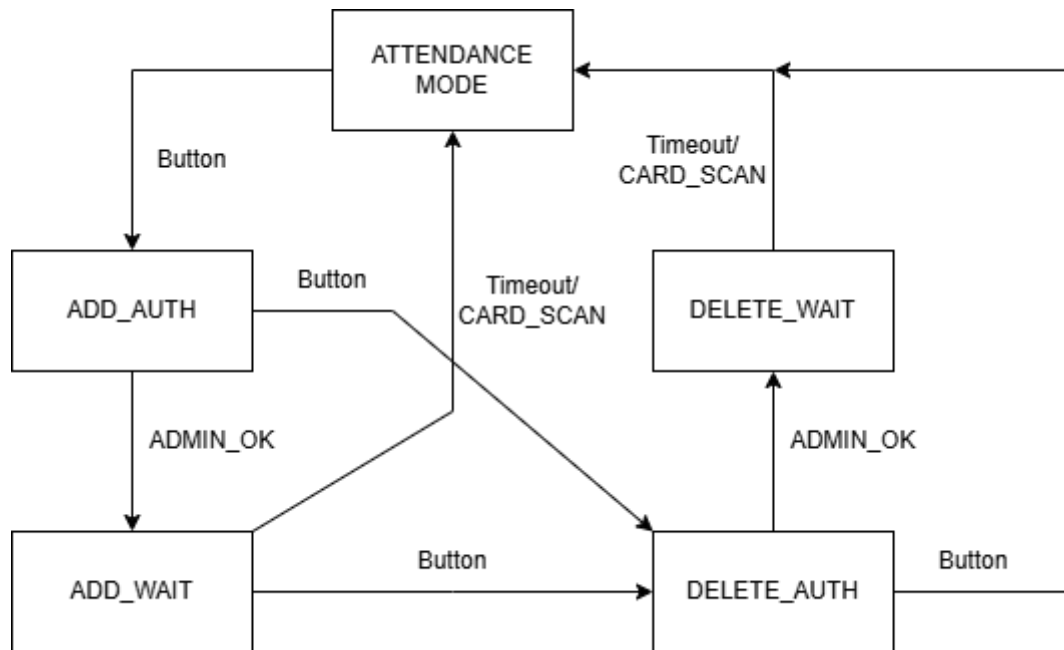


Figure 7: Administrator mode state machine

The administrator state machine includes five main states:

- **NORMAL:** Default attendance mode.
- **ADD_AUTH:** Waiting for admin authentication before adding a card.
- **ADD_WAIT:** Waiting for the target card to be added.
- **DELETE_AUTH:** Waiting for admin authentication before deleting a card.
- **DELETE_WAIT:** Waiting for the target card to be deleted.

The state transition logic is summarized in Table 6.

Table 6: Administrator mode state description

State	Event	Next Action
NORMAL	Button pressed once	Enter ADD_AUTH state.
ADD_AUTH	Admin card scanned successfully	Enter ADD_WAIT state.
ADD_WAIT	Target card scanned	Add UID to Flash database, then return to NORMAL.
NORMAL / ADD_AUTH	Button pressed again	Enter DELETE_AUTH state.
DELETE_AUTH	Admin card scanned successfully	Enter DELETE_WAIT state.
DELETE_WAIT	Target card scanned	Remove UID from Flash database, then return to NORMAL.
Any AUTH state	Timeout after 5 seconds	Return to NORMAL.
Any WAIT state	Timeout after 10 seconds	Return to NORMAL.

The timeout mechanism is important because it prevents the system from remaining in administrator mode indefinitely. This improves both safety and usability.

6.4. Attendance Logic

The attendance logic determines whether a user is entering or leaving. A simple method is to store the latest state of each UID. If the user is currently marked as OUT, the next valid scan is considered IN. If the user is currently marked as IN, the next valid scan is considered OUT.

This logic can be represented as:

$$State_{new} = \begin{cases} IN, & \text{if } State_{old} = OUT \\ OUT, & \text{if } State_{old} = IN \end{cases} \quad (7)$$

After the state is updated, the system displays the result and plays either a greeting or goodbye audio message.

7. Implementation Details

7.1. RFID Reading

The RFID module is responsible for detecting a card and reading its UID. The firmware periodically polls the MFRC522 module. When a new card is detected, the UID bytes are copied into a buffer and passed to the student database module for lookup.

The UID is treated as the primary identification key. To avoid duplicated operations, the firmware should prevent repeated scanning of the same card within a very short interval. This can be done by using a cooldown time or by waiting until the card is removed before accepting another scan.

7.2. UID Database in Flash

The UID database is stored in internal Flash memory. According to the design, Flash sector 7 is used for the UID database. The database contains a header and a list of UID records.

A possible Flash database structure is:

```
1 typedef struct {  
2     uint32_t magic;  
3     uint32_t count;  
4     uid_record_t records[MAX_CARDS];  
5     uint32_t checksum;  
6 } flash_db_t;
```

The `magic` field is used to check whether the Flash database has been initialized. The `count` field stores the number of valid cards. The `records` array stores UID entries. The `checksum` field can be used to detect data corruption.

7.3. RAM Cache

Although Flash memory provides persistent storage, reading and searching directly from Flash every time is not ideal. Therefore, the UID database is loaded into RAM during initialization.

The advantage of RAM caching is that UID lookup becomes faster. Based on the group measurement, UID lookup using RAM cache takes less than 1 ms. This is suitable for real-time attendance interaction.

7.4. Atomic Flash Update

Flash memory cannot be overwritten byte by byte like RAM. Usually, a Flash sector must be erased before new data is written. Therefore, adding or deleting a UID requires an erase-modify-write operation.

To improve reliability, the system should use an atomic update strategy:

1. Load the current database from Flash into RAM.
2. Modify the RAM copy by adding or deleting the UID.
3. Validate the new database.
4. Erase the Flash sector.
5. Write the new database to Flash.
6. Verify the written data.
7. Roll back or report an error if verification fails.

This method reduces the risk of storing an invalid database. However, if power is lost during Flash update, data corruption is still possible. A more advanced design may use two Flash sectors and a version flag to support full rollback.

7.5. OLED User Interface

The OLED display is used to show system status. Typical messages include:

- System ready.
- Card detected.
- Access accepted.
- Unknown card.

- Check-in successful.
- Check-out successful.
- Add mode.
- Delete mode.
- Admin authentication failed.

The OLED user interface improves the clarity of the system. It also helps debugging because the current state can be displayed directly.

7.6. RTC Timestamping

The DS3231 RTC provides the current date and time. When a valid attendance event occurs, the system reads the RTC through I2C and associates the timestamp with the attendance event.

A typical timestamp format is:

$$YYYY - MM - DD \quad HH : MM : SS \quad (8)$$

Using an RTC is more reliable than relying only on the microcontroller internal timer because the RTC can continue keeping time even when the main system is powered off, provided that the backup battery is installed.

7.7. Voice Feedback

Voice feedback is implemented using I2S audio transmission. Audio data is stored as C header arrays, such as `in_wav.h` and `out_wav.h`. When an attendance event occurs, the application calls the audio player module. The audio player then uses the I2S BSP module to transmit audio samples through DMA.

Using DMA is important because it allows audio playback to continue while the CPU performs other tasks. This prevents the system from being blocked during audio playback.

7.8. Button Debounce

Mechanical buttons often generate bouncing signals when pressed or released. If the firmware reads the GPIO directly, one physical press may be interpreted as multiple presses. Therefore, the button module includes debounce logic and edge-trigger detection.

The button module should only generate one event for one valid press. This is important because the button is used to change administrator mode. Without debounce, the system may accidentally skip states.

8. Results and Evaluation

8.1. Implemented Features

The main implemented features are summarized in Table 7.

Table 7: Implemented technical features

Feature	Module	Status
Persistent UID storage	flash_db	Implemented
Magic number validation	flash_db	Implemented
Atomic erase-modify-write	flash_db	Implemented
RAM cache for fast UID lookup	flash_db, student_db	Implemented
RFID card reading	rfid	Implemented
OLED display feedback	ui_oled	Implemented
RTC time reading	rtc_ds3231	Implemented
Audio playback using DMA	i2s_audio	Implemented
Admin authentication	admin_mode	Implemented
Button debounce and edge trigger	button	Implemented

8.2. Memory Usage

The memory usage of the system is summarized in Table 8.

Table 8: Memory usage

Item	Value
Total C modules	10
Total code lines, excluding HAL and third-party libraries	Approximately 1500 lines
Code Flash usage	Approximately 30 KB / 512 KB
Audio data	Approximately 35 KB
RAM usage	Approximately 3 KB / 128 KB
UID database capacity	Up to approximately 200 cards

The memory usage is acceptable for the STM32F411CEU6. The firmware and audio

resources occupy only a small portion of the available Flash memory. RAM usage is also low, leaving sufficient memory for stack, heap, and future features.

8.3. Timing Performance

The timing performance is summarized in Table 9.

Table 9: Timing performance

Operation	Estimated / Measured Time
UID lookup using RAM cache	Less than 1 ms
Flash erase and rewrite	Approximately 150 ms
Audio playback latency	Less than 50 ms
Admin authentication timeout	5 seconds
Admin waiting timeout	10 seconds

The UID lookup speed is fast enough for real-time attendance checking. Flash update is slower because Flash erase and write operations naturally take more time. However, Flash update only occurs during administrator add/delete operations, not during every normal attendance scan. Therefore, it does not significantly affect the normal user experience.

8.4. Requirement Satisfaction

Table 10 shows how the design satisfies the project requirements.

Table 10: Requirement satisfaction

Requirement	How the System Satisfies It
Layered architecture	The firmware is divided into HAL/third-party, BSP, application, and main state-machine layers.
Design for reuse	BSP drivers are separated from application logic.
Single responsibility	Each module performs one major function.
Encapsulation	Internal helper functions can be declared static, while only required APIs are exposed in header files.
Multiple peripherals	The system integrates SPI, I2C, I2S, GPIO, Flash, RTC, and DMA.
Persistent data	UID database is stored in internal Flash sector 7.
User feedback	OLED display and voice audio are provided.
Security	Admin card authentication is required before adding or deleting cards.

8.5. Overall Assessment

Overall, the system satisfies the main requirements of an embedded RFID attendance system. It can identify RFID cards, verify valid users, provide real-time feedback, store the UID database persistently, and support administrator operations.

The design is also practical because it uses low-cost and widely available modules. The STM32F411CEU6 provides enough Flash, RAM, and communication peripherals for this application. The four-layer firmware architecture makes the project easier to extend and maintain.

9. Discussion

9.1. Advantages

The proposed system has several advantages.

First, the system is compact and low-cost. It uses common embedded modules such as MFRC522, SSD1306 OLED, DS3231 RTC, and STM32 Black Pill. These components are easy to obtain and suitable for student projects.

Second, the system provides both visual and audio feedback. The OLED display shows detailed information, while voice feedback provides immediate confirmation to the user.

Third, the system supports persistent storage. Because the UID database is stored in internal Flash memory, registered cards are preserved after reset or power loss.

Fourth, the system has a clear software architecture. By separating BSP drivers from application logic, the firmware is easier to debug and reuse.

Fifth, administrator authentication improves security. Users cannot freely add or delete cards unless they have access to the admin card.

9.2. Limitations

Although the system works well for small-scale applications, it still has several limitations.

First, the UID database capacity is limited by the reserved Flash sector size. The current design supports approximately 200 cards, which is enough for a classroom or small laboratory but may not be sufficient for large organizations.

Second, internal Flash memory has a limited erase/write lifetime. Frequent add/delete operations may reduce Flash endurance. However, in a typical attendance system, card registration is not performed frequently, so this limitation is acceptable.

Third, the current system stores and checks UID values. UID-based authentication is simple but not highly secure against advanced cloning attacks. For higher security, cryptographic authentication should be considered.

Fourth, the current design is mainly local. Without network connectivity, attendance data cannot be automatically uploaded to a server or cloud database.

Fifth, if power is lost during Flash update, database corruption may occur unless a robust backup mechanism is implemented.

9.3. Practical Applicability

The system is suitable for small to medium applications such as classroom attendance, laboratory access tracking, club attendance management, and small office check-in systems. The design is also valuable as an educational embedded system project because it integrates many important microcontroller topics, including SPI, I2C, I2S, DMA, Flash programming, state machines, and modular firmware design.

9.4. Possible Improvements

Several improvements can be made in future versions:

- Add MicroSD logging for long-term attendance records.
- Add Wi-Fi or Ethernet communication for cloud synchronization.
- Use two Flash sectors for safer database update and rollback.
- Add password or keypad authentication for administrator mode.
- Use more secure RFID cards with cryptographic authentication.
- Add PC software or a web dashboard for attendance management.
- Add battery backup for the whole system.
- Add enclosure design for real-world deployment.

10. Conclusion

This report presented the design and analysis of an RFID Attendance System based on the STM32F411CEU6 microcontroller. The system integrates an MFRC522 RFID reader, SSD1306 OLED display, DS3231 RTC module, I2S audio amplifier, button input, and internal Flash memory.

The system can read RFID card UIDs, verify users through a persistent UID database, determine attendance status, display information on an OLED screen, and provide voice feedback. The administrator mode allows authorized adding and deleting of RFID cards using a state-machine approach.

The firmware is organized into four layers: HAL and third-party resources, BSP driver wrappers, application logic, and main state-machine routing. This architecture improves modularity, maintainability, and code reuse. The system also demonstrates efficient use of memory, with approximately 30 KB of code Flash usage, 35 KB of audio data, and only about 3 KB of RAM usage.

In conclusion, the project successfully demonstrates a practical embedded attendance system. It is compact, low-cost, modular, and suitable for educational or small-scale real-world applications. Future work can focus on improving security, adding external data logging, supporting network synchronization, and enhancing database reliability.

References

- [1] STMicroelectronics, *STM32F411xC STM32F411xE Datasheet: Arm Cortex-M4 32-bit MCU with FPU*. Available: <https://www.st.com/resource/en/datasheet/stm32f411ce.pdf>. Accessed: May 2026.
- [2] NXP Semiconductors, *MFRC522 Standard Performance MIFARE and NTAG Frontend Datasheet*. Available: <https://www.nxp.com/docs/en/data-sheet/MFRC522.pdf>. Accessed: May 2026.
- [3] Analog Devices, *DS3231 Extremely Accurate I2C-Integrated RTC/TCXO/Crystal Datasheet*. Available: <https://www.analog.com/media/en/technical-documentation/data-sheets/ds3231.pdf>. Accessed: May 2026.
- [4] Solomon Systech, *SSD1306 OLED/PLED Segment/Common Driver with Controller Datasheet*. Available: <https://cdn-shop.adafruit.com/datasheets/SSD1306.pdf>. Accessed: May 2026.
- [5] Analog Devices, *MAX98357A/MAX98357B PCM Input Class D Audio Power Amplifiers*. Available: <https://www.analog.com/media/en/technical-documentation/data-sheets/MAX98357A-MAX98357B.pdf>. Accessed: May 2026.

A. Appendix A: Suggested Figure Files

To compile this report in Overleaf, upload the project images with the following names:

Table 11: Suggested image file names

File Name	Content
block_diagram.png	Overall block diagram with STM32F411CEU6, RFID, OLED, RTC, and MicroSD.
power_budget.png	Power supply and current estimation diagram.
schematic.png	Full system schematic.
admin_fsm.png	Administrator mode state machine diagram.

B. Appendix B: Main Pseudocode

Listing 1: Main application pseudocode

```
1 int main(void)
2 {
3     HAL_Init();
4     SystemClock_Config();
5
6     GPIO_Init();
7     SPI_Init();
8     I2C_Init();
9     I2S_DMA_Init();
10
11    RFID_Init();
12    OLED_Init();
13    RTC_DS3231_Init();
14    FlashDB_Init();
15    Button_Init();
16    AudioPlayer_Init();
17
18    FlashDB_LoadToRamCache();
19
20    OLED_ShowMessage("System Ready");
```

```
21
22     while (1)
23     {
24         Button_Update();
25
26         if (Button_WasPressed())
27         {
28             AdminMode_HandleButton();
29         }
30
31         if (RFID_IsCardPresent())
32         {
33             uid_t uid = RFID_ReadUID();
34
35             if (AdminMode_IsActive())
36             {
37                 AdminMode_HandleCard(uid);
38             }
39             else
40             {
41                 Attendance_HandleCard(uid);
42             }
43         }
44
45         AudioPlayer_Update();
46     }
47 }
```

C. Appendix C: Admin Mode Pseudocode

Listing 2: Administrator mode pseudocode

```
1 void AdminMode_HandleCard(uid_t uid)
2 {
3     switch (admin_state)
4     {
5         case ADD_AUTH:
```

```
6         if (StudentDB_IsAdmin(uid))
7         {
8             admin_state = ADD_WAIT;
9             OLED_ShowMessage("Scan card to add");
10        }
11        else
12        {
13            OLED_ShowMessage("Admin denied");
14            admin_state = NORMAL;
15        }
16        break;
17
18    case ADD_WAIT:
19        FlashDB_AddUID(uid);
20        FlashDB_LoadToRamCache();
21        OLED_ShowMessage("Card added");
22        admin_state = NORMAL;
23        break;
24
25    case DELETE_AUTH:
26        if (StudentDB_IsAdmin(uid))
27        {
28            admin_state = DELETE_WAIT;
29            OLED_ShowMessage("Scan card to delete");
30        }
31        else
32        {
33            OLED_ShowMessage("Admin denied");
34            admin_state = NORMAL;
35        }
36        break;
37
38    case DELETE_WAIT:
39        FlashDB_DeleteUID(uid);
40        FlashDB_LoadToRamCache();
41        OLED_ShowMessage("Card deleted");
42        admin_state = NORMAL;
43        break;
```

```
44  
45     default:  
46         break;  
47     }  
48 }
```